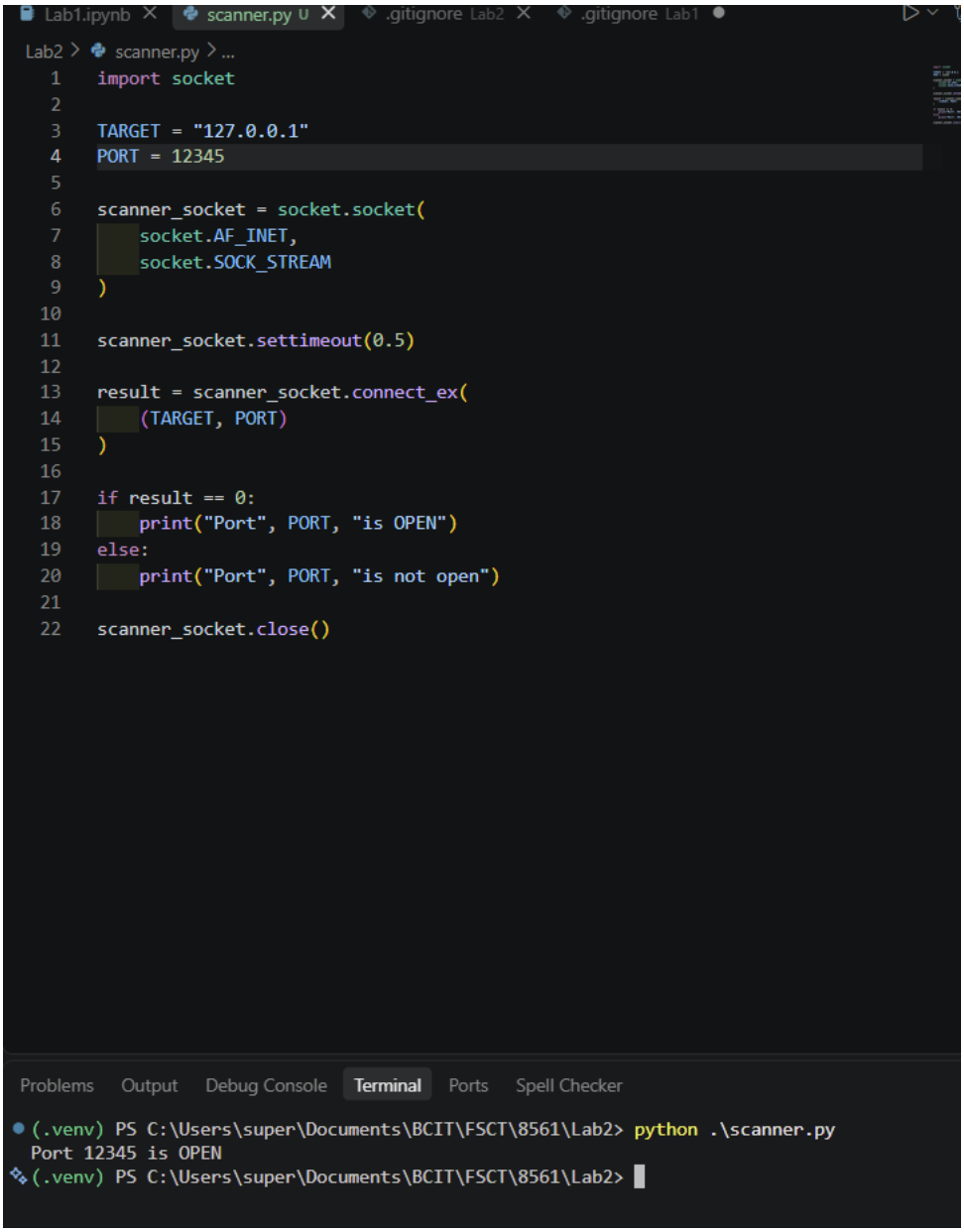
Braeden Venne

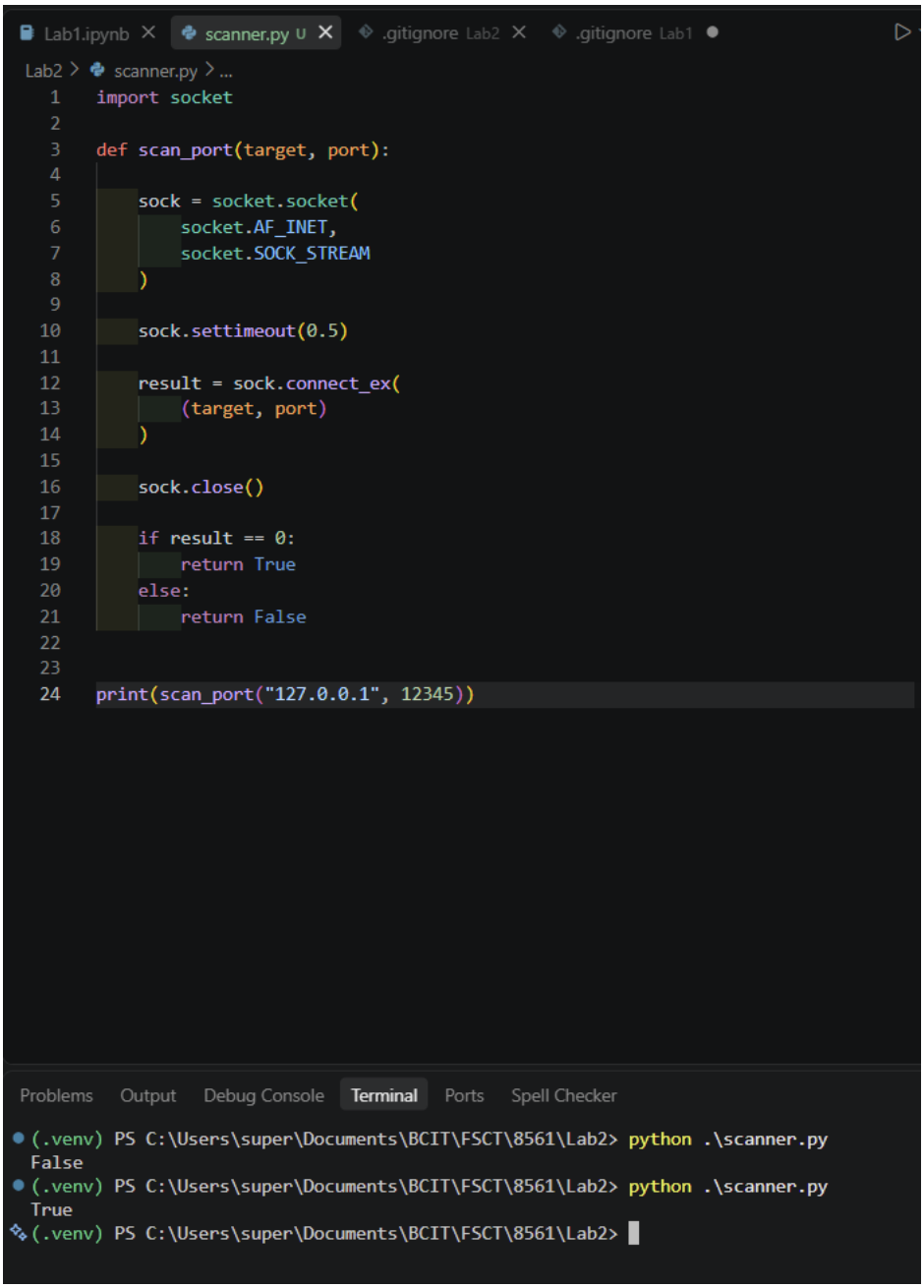
Lab 2

Part 3 – Test One Port First



Braeden Venne

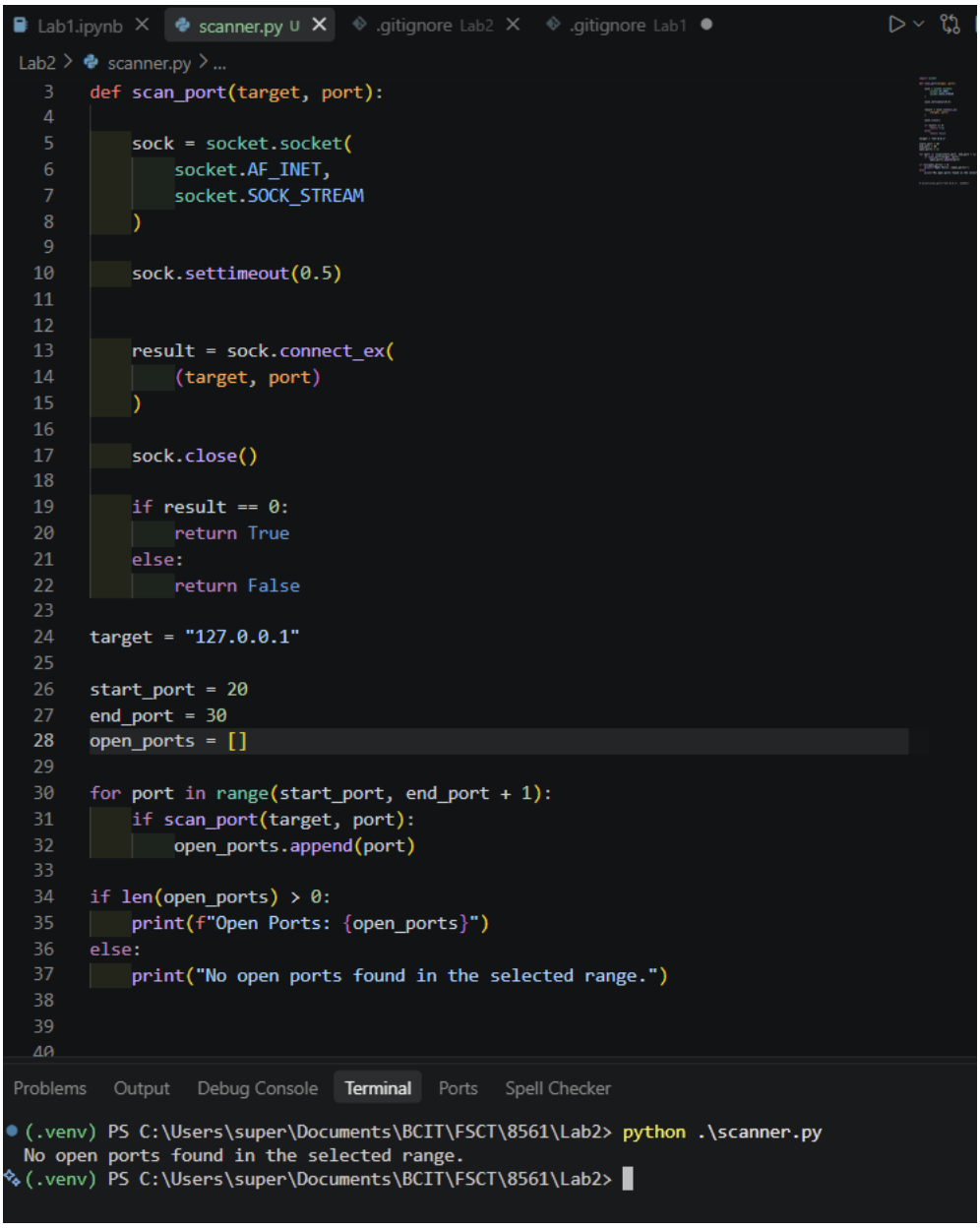
Part 4 – Turn one port test into a function



- Tested with server off first -> Program returned False
- Tested with server on after -> Program returned True

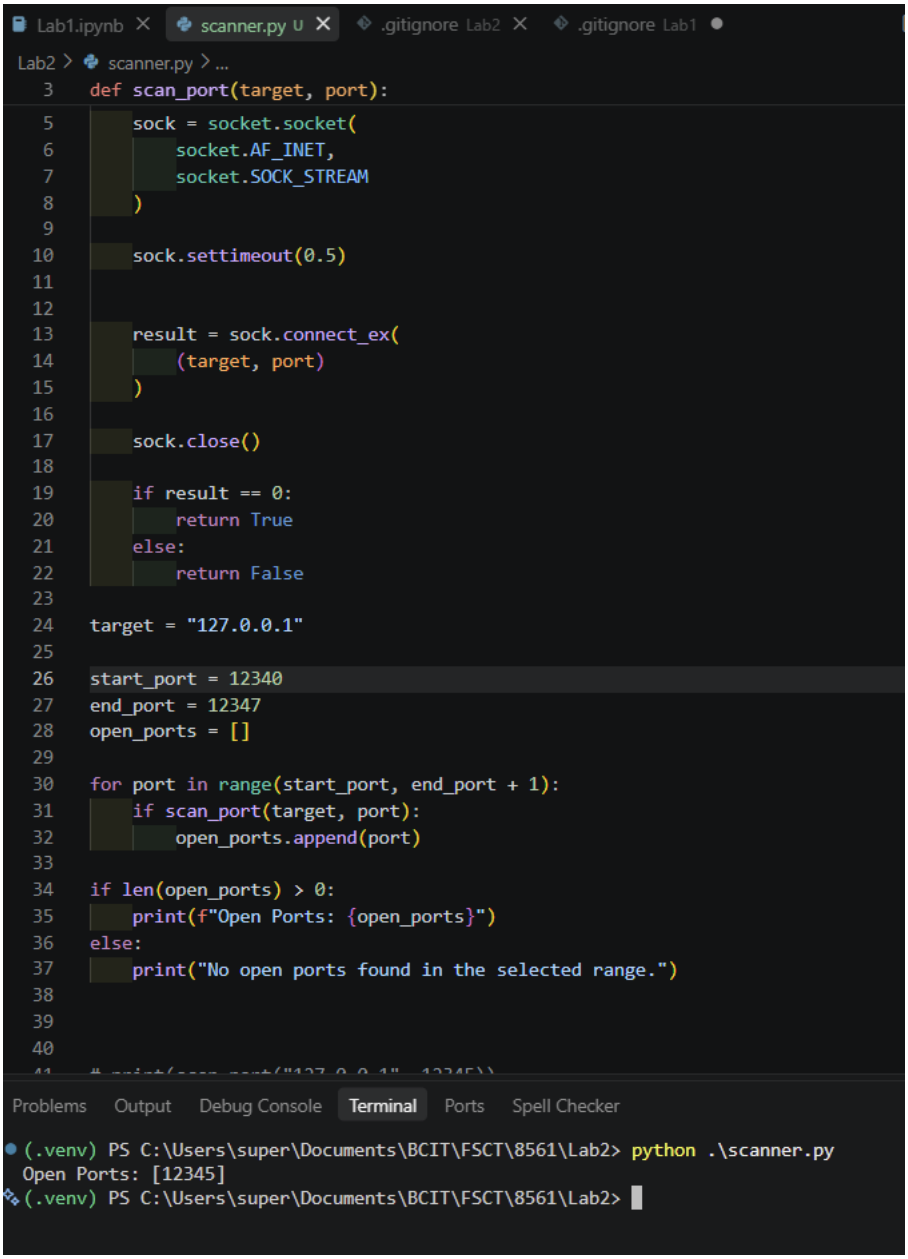
Braeden Venne

Part 5 – Scan a Small Range of Ports



- Tested with no open ports first, returned the no open ports found message as expected

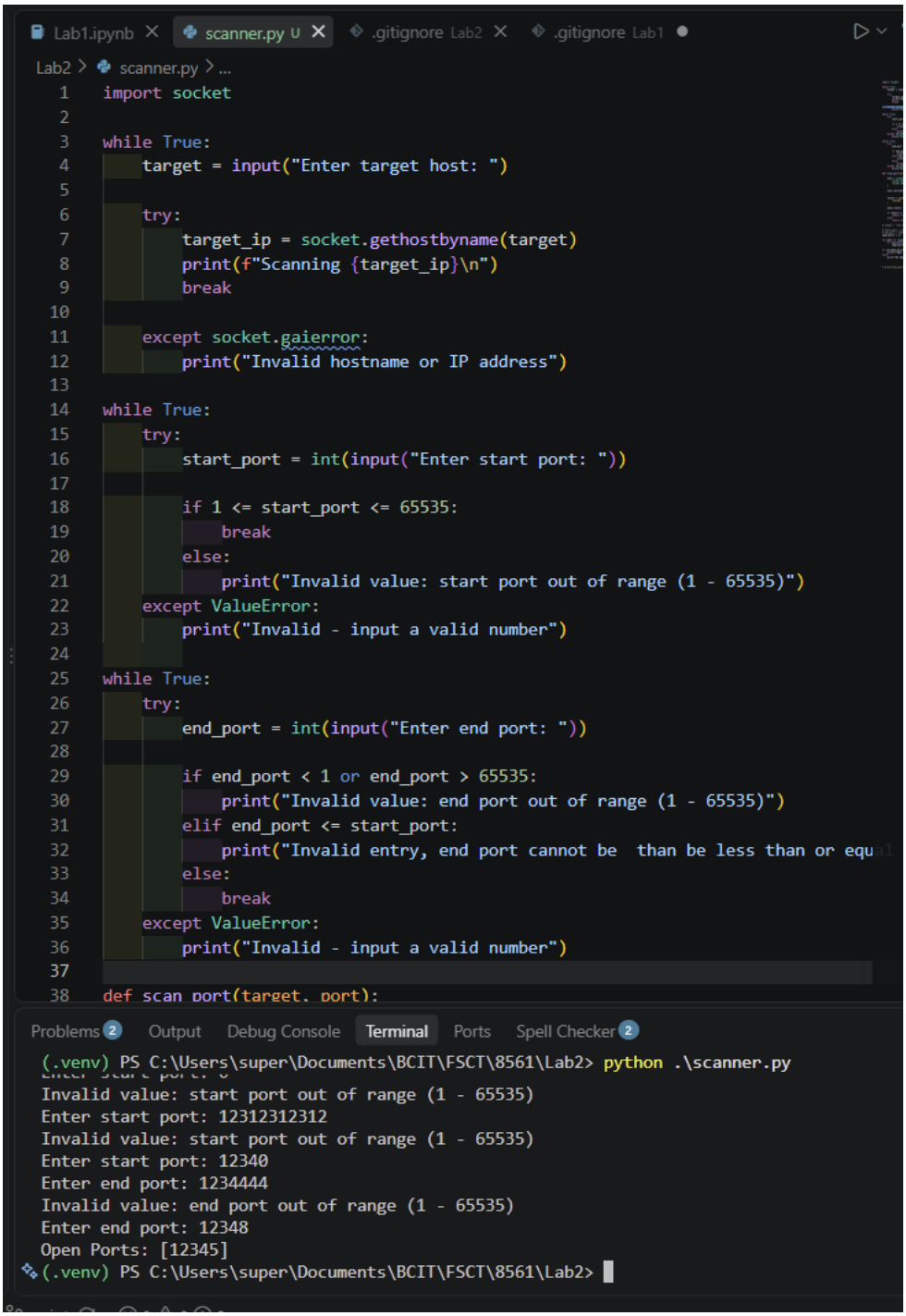
Braeden Venne



- Works as intended while my Lab1 server is running

Braeden Venne

Part 7 + Part 8 – Accept User Input and Validate the Port Range



- Added while loops for input validation
- Tested various incorrect inputs, program works as intended

Braeden Venne

Part 9 – Build the Complete Scanner

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2> python .\scanner.py
Enter target host: 127.0.0.1
Scanning 127.0.0.1

Enter start port: 20
Enter end port: 30
Target: 127.0.0.1

scanning TCP ports 20 - 30

PORT            STATE           SERVICE
22              Open            SSH
27              Open            N/A
20              Not Open        N/A
21              Not Open        N/A
23              Not Open        N/A
24              Not Open        N/A
25              Not Open        N/A
26              Not Open        N/A
28              Not Open        N/A
29              Not Open        N/A
30              Not Open        N/A

Scan complete.
2 open port(s) found.
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2>
```

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2> python .\scanner.py
Enter target host: 127.0.0.1
Scanning 127.0.0.1

Enter start port: 80
Enter end port: 90
Target: 127.0.0.1

scanning TCP ports 80 - 90

PORT            STATE           SERVICE
80              Open            HTTP
90              Open            N/A
81              Not Open        N/A
82              Not Open        N/A
83              Not Open        N/A
84              Not Open        N/A
85              Not Open        N/A
86              Not Open        N/A
87              Not Open        N/A
88              Not Open        N/A
89              Not Open        N/A

Scan complete.
2 open port(s) found.
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2>
```

Braeden Venne

Part 10 – Robustness Test

1. A known open port
    a. The screenshots demonstrate that ssh and http are recognized

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2> python .\scanner.py
Enter target host: 127.0.0.1
Scanning 127.0.0.1

Enter start port: 20
Enter end port: 30
Target: 127.0.0.1

scanning TCP ports 20 - 30

PORT            STATE           SERVICE
22              Open            SSH
27              Open            N/A
20              Not Open        N/A
21              Not Open        N/A
23              Not Open        N/A
24              Not Open        N/A
25              Not Open        N/A
26              Not Open        N/A
28              Not Open        N/A
29              Not Open        N/A
30              Not Open        N/A

Scan complete.
2 open port(s) found.
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2>
```

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2> python .\scanner.py
Enter target host: 127.0.0.1
Scanning 127.0.0.1

Enter start port: 80
Enter end port: 90
Target: 127.0.0.1

scanning TCP ports 80 - 90

PORT            STATE           SERVICE
80              Open            HTTP
90              Open            N/A
81              Not Open        N/A
82              Not Open        N/A
83              Not Open        N/A
84              Not Open        N/A
85              Not Open        N/A
86              Not Open        N/A
87              Not Open        N/A
88              Not Open        N/A
89              Not Open        N/A

Scan complete.
2 open port(s) found.
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2>
```

2. Invalid Hostname

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2> python .\scanner.py
Enter target host: 127
Invalid hostname or IP address
Enter target host:
```

3. Port below 1 or above 65535

```
Enter start port: 0
Invalid value: start port out of range (1 - 65535)
Enter start port:
```

```
Enter start port: 65536
Invalid value: start port out of range (1 - 65535)
Enter start port:
```

Braeden Venne

4. End port is less than or equal to start port

```
Enter start port: 20
Enter end port: 20
Invalid entry, end port cannot be  than be less than or equal to start port
Enter end port:
```

```
Enter end port: 19
Invalid entry, end port cannot be  than be less than or equal to start port
Enter end port:
```

5. Non-numeric port

```
Enter end port: abc
Invalid - input a valid number
Enter end port:
```

6. No ports in selected range

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2> python .\scanner.py
Scanning 127.0.0.1

Enter start port: 19
Enter end port: 20
Target: 127.0.0.1

scanning TCP ports 19 - 20

No open ports found in the selected range.
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2>
```

7. Not opened port

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2> python .\scanner.py
Enter target host: 127.0.0.1
Scanning 127.0.0.1

Enter start port: 80
Enter end port: 90
Target: 127.0.0.1

scanning TCP ports 80 - 90

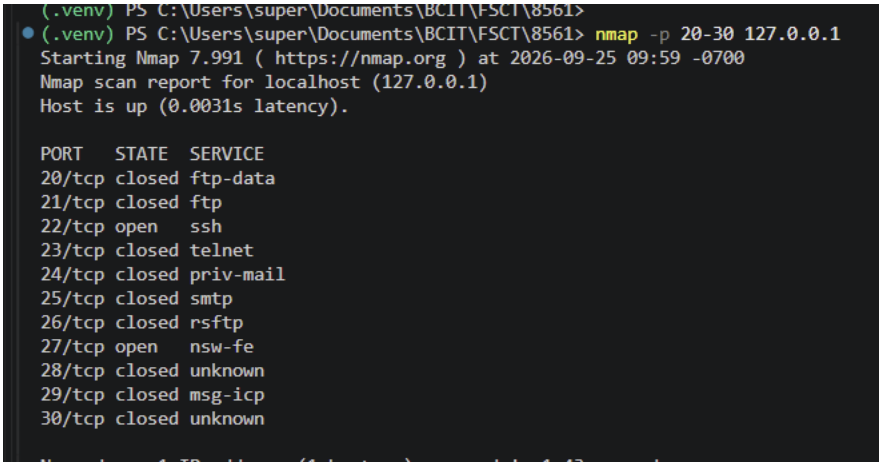
PORT            STATE           SERVICE
80              Open            HTTP
90              Open            N/A
81              Not Open        N/A
82              Not Open        N/A
83              Not Open        N/A
84              Not Open        N/A
85              Not Open        N/A
86              Not Open        N/A
87              Not Open        N/A
88              Not Open        N/A
89              Not Open        N/A

Scan complete.
2 open port(s) found.
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab2>
```

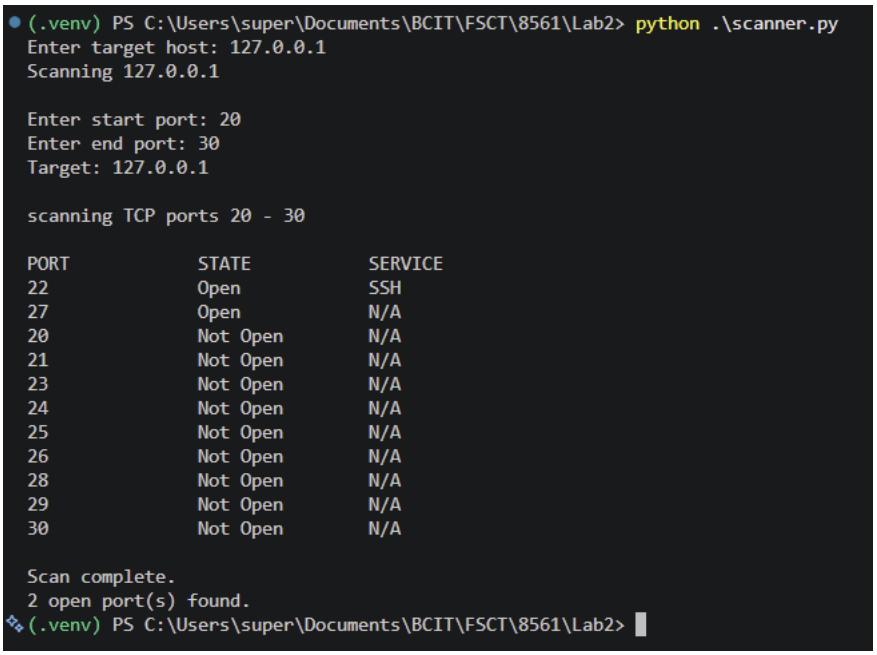
Braeden Venne

Part 11 – Compare Scanner with Nmap

Nmap results for ports 20 - 30:



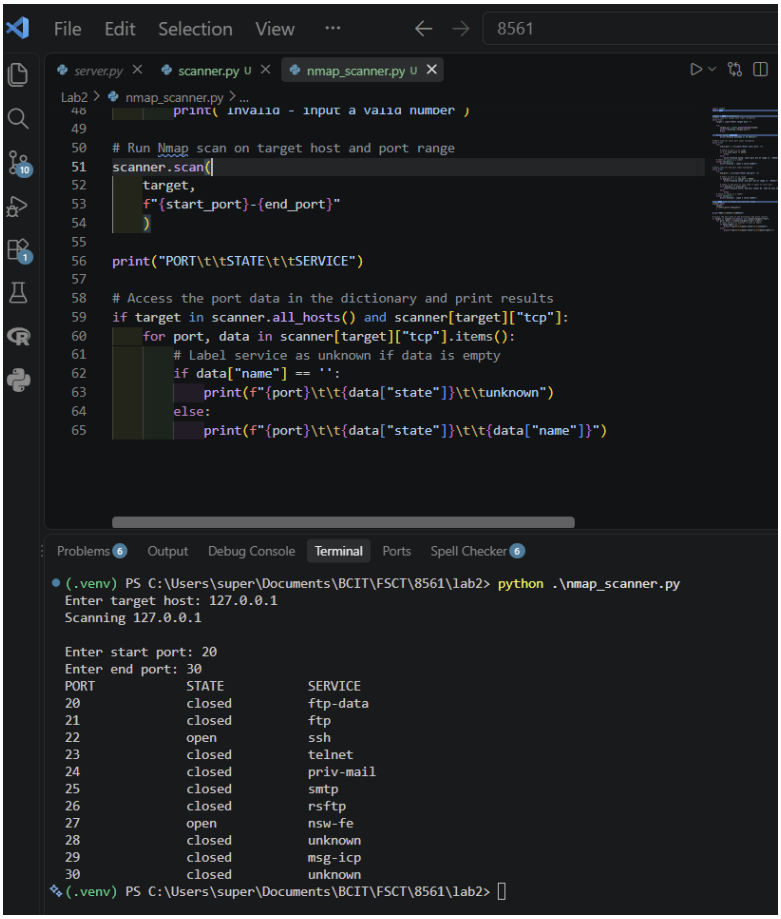
Scanner results for ports 20 - 30:



1. Did Nmap identify the same open ports as your Python scanner?
    a. Yes, both scans were able to detect ports 22 and 27
2. What additional information did Nmap provide?
    a. Nmap reported that the “Host is up” alongside with its response time (0.0031s latency)
    b. It also identified the transport layer protocol for each port (i.e. 22/tcp, 27/tcp, etc.).
    c. It also had higher level mapping of ports to their well-known service names whereas majority in my script were labelled “N/A”.

3. Did Nmap distinguish states that your scanner could not?
    a. Yes, Nmap explicitly labeled ports as closed because it was able to receive a TCP RST packet back from the host. My scanner on the other hand labels these ports as “Not open” as it cannot reliably tell the difference between a port that is explicitly closed versus one that is filtered.
4. Why is your Python scanner useful for learning even though Nmap is more capable?
    a. Writing a scanner from scratch is useful for showing you line-by-line how TCP port scanning works, whereas Nmap does all the heavy lifting for you.

Part 12 – Rebuild Python Scanner Using python-nmap



1. Which version required more code?
    a. My original scanner.py requires more code than my nmap_scanner.py.
2. Which version provided more information?
    a. Nmap_scanner.py provides more information than scanner.py.
3. What information did Nmap provide that your raw socket scanner did not?
    a. Nmap can reliably tell us when a port is “closed” rather than just “Not open” and it has more information on the most-likely services the detected ports belong to.
4. What work is `python-nmap` hiding from the programmer?

   a. Python-nmap hides a lot of the manual coding work behind the single scanner.scan() function call. In my scanner.py code I had to write all the underlying network logic myself. I also had to write a separate for loop with a try/except block using socket.getservbyport() just to look up the standard service names and even then, it could not identify as many as Nmap could.
5. Why was it useful to build the raw socket version **before** using python-nmap?
   a. It was useful to build the raw socket version before using python-nmap as the scanner I built from scratch gave me a line-by-line understanding of how TCP port scanning works. Python-nmap, on the other hand, did majority of the heavy lifting for me.
6. If you were developing a real security application, which approach would you choose and why?
   a. If I were developing a real security application, I would choose to use python-nmap. Nmap provides far greater reliability, speed, and a more detailed analysis, which are essential for a production=level security tool. Additionally, it is also a lot simpler to use which would save a significant amount of time and resources compared to building and maintaining a custom raw socket scanner.

Part 13 – Security Reflection

1. What information can an attacker learn from port scanning?
   a. An attacker can gain information on the active hosts on the network, which TCP/UDP ports are open, the services and app versions running on those ports, and the host's operating system.
2. Why can an open port increase a system's attack surface?
   a. An open port means that a network service is actively listening and accepting incoming traffic. With each open service there is the potential for software bugs, configuration errors, and vulnerabilities that an attacker could exploit to gain access.
3. Does an open port automatically mean that a vulnerability exists? Explain.
   a. No, an open port only means that the service is actively listening and accepting connections. Vulnerabilities only exist if the software listening on that specific port contains a software bug, misconfiguration, or utilizes weak credentials.
4. Why should unnecessary network services be disabled or restricted?
   a. Disabling or restricting unnecessary network services minimizes the system's attack surface and reduces the amount of entry points that may introduce a risk of exploitation.
5. What is the difference between an open port and a closed port?
   a. An open port has an application actively listening and accepting connections and will respond with a TCP SYN-ACK.
   b. A closed port is accessible, however there is no application listening on it, which causes the target host to respond with a TCP RST flag.
6. Why might a firewall make scan results difficult to interpret?

    a. Since firewalls can block, drop, or manipulate packets the scanner cannot reliably tell if the port is open, closed, or filtered by the firewall.
7. Why can our simple TCP Connect scanner not reliably identify every filtered port?
    a. Our simple TCP Connect scanner relies on connect_ex(), which only returns a binary success or failure. If a port fails to connect, it does not tell us specifically whether a host actively rejected the connection or dropped it because the firewall filtered it, it only returns the same generic failure code.
8. How could defenders detect or limit scanning activity?
    a. Defenders could use intrusion detection systems, such as Snort, to flag rapid connection attempts across multiple ports. For limiting scanning, they can apply rate limiting on firewalls.
9. Why should defenders scan their own systems?
    a. Defenders should scan their own networks to find exposed services and misconfigurations before bad actors do. It can also help verify that configured firewall rules are working as intended, detect unauthorized rogue services, and ensure new deployments are secure.
10. Why is authorization important before performing network scanning?
    a. Authorization is important because unauthorized scans look like active attacks and can trigger intrusion detection systems or automated blocks. Authorized scans also avoid triggering legal/disciplinary issues.

Part 14 – Security Analysis

Based on the scan results from this lab, the authorized target was localhost (127.0.0.1). In this lab, I tested multiple port ranges, including ports 20 – 30 and 80 – 90. Across these ranges, the scanner I built was successful in identifying open ports on 22, 27, 80, and 90. Among these ports, my scan correctly identified port 22 as SSH and port 80 as HTTP. On the other hand, ports 27 and 90 were both labelled as “N/A”.

Comparing the Python scanner to Nmap highlights notable key differences. The custom Python script utilizes raw TCP connect_ex() sockets, which only return a generic binary success/failure and label unassigned ports strictly as “Not Open”. It could not reliably determine whether a port was closed or filtered; it also could not identify many higher-level service maps beyond standard lookups. Nmap, on the other hand, can reliably determine when a port is closed or filtered since it gains this information from returned TCP RST packets. Nmap was also able to correctly map more services to their well-known counterparts (i.e., identifying port 27 as nsw-fe and port 20 as ftp-data).

From a defender’s perspective, each open port represents an exposed listening service that expands the host’s overall attack surface. Unnecessary exposed services create significant operational risks. If we have software running on ports HTTP or SSH and they contain software bugs, misconfigurations, or weak credentials, an attacker can use exploit them and potentially gain unauthorized access into the system or compromise system integrity. Exposed non-standard ports

also present security risks as they are often forgotten background services or unauthorized applications running without any monitoring.

To mitigate risks and reduce unnecessary exposure, defenders should implement strict defensive measures. Unnecessary network services and open ports should be disabled or closed to reduce the attack surface. Firewall rules should be configured to enforce the principle of least privilege, restricting access to essential ports via whitelisting. Finally, regular internal port scans should be conducted to detect rogue services before bad actors do.